A Multi-Dimensional Priority Model for Heterogeneous DAG Workloads in Distributed Computing Continuums

Authors
Affiliation

***Abstract*—Distributed meta-orchestration platforms coordinate computational workloads across heterogeneous infrastructure spanning cloud, edge, and HPC systems. These platforms must schedule diverse workloads—from real-time safety-critical mine seismology to best-effort research analytics—on shared executor pools. Existing schedulers rely on a single integer priority, which conflates urgency, importance, fairness, and domain-specific value into one dimension, leading to priority inversions and starvation.**

We analyze eight realistic workload types—including a real-time IoT edge scenario requiring preemption of lower-priority work—identify common priority dimensions, and evaluate five scheduling approaches: weighted sum models, rule-based systems, lexicographic ordering, constraint-based optimization, and machine learning methods. Based on this analysis, we propose a hybrid priority model combining lexicographic urgency classes with weighted multi-dimensional scoring within each class. The model supports static, dynamic, and DAG-dependent priority dimensions while maintaining structural guarantees for safety-critical workloads and multi-tenant fairness. We describe the model formally, show how it instantiates for each workload type, and discuss its properties with respect to preemption, starvation freedom, interpretability, and cross-environment scheduling.

***Index Terms*—distributed scheduling, DAG workflows, priority models, computing continuums, meta-orchestration**

I. Introduction

Computing continuums [1] integrate heterogeneous resources—cloud data centers, HPC clusters, edge devices, and IoT infrastructure—into unified execution environments. Meta-orchestration platforms such as ColonyOS [2] coordinate workloads across these environments by acting as brokers between users who declare *what* to compute and distributed executors that know *how* to execute on their specific platforms.

Workloads in these systems are expressed as directed acyclic graphs (DAGs) of processes, where each process specifies a function to execute, its arguments, resource requirements, and dependencies on other processes. The scheduler assigns waiting processes to available executors based on priority.

The fundamental challenge is that these platforms serve radically different workloads concurrently: a mine seismology system producing continuous streams of safety-critical events, a satellite surveillance service triggered by new imagery, an agentic AI system dynamically growing its DAG, a multi-tenant ETL platform with SLA deadlines, infrastructure automation workflows, cross-environment HPC training pipelines, large-scale parameter sweeps, and real-time IoT edge systems where safety-critical anomaly response must preempt normal monitoring on resource-constrained edge gateways. Each workload has its own notion of what "important" means, yet they all compete for shared executor resources.

Current systems assign priority as a single integer at submission time. This approach fails in three fundamental ways:

1) **Priority is multi-dimensional.** A seismic event's importance depends on the number of sensors that triggered, the mine region, and the event classification—not a single number.
2) **Priority is dynamic.** An ETL pipeline's urgency increases as its SLA deadline approaches. A drug screening compound's priority depends on docking results from an earlier DAG step.
3) **Priority is contextual.** A cloud burst workflow's urgency depends on another workflow's queue depth. Safety-critical work must structurally guarantee preemption over best-effort research. On resource-constrained edge devices, an emergency anomaly response must be able to preempt running lower-priority monitoring processes.

In this paper, we make the following contributions:

- We present eight realistic, detailed use cases spanning the diversity of workloads in computing continuums, including a real-time IoT scenario requiring preemption (Section III).
- We extract common priority dimensions from these use cases and identify requirements—including preemption support—that any priority model must satisfy (Section IV).
- We evaluate five alternative scheduling approaches and assess their suitability (Section V).
- We propose a hybrid priority model combining lexicographic urgency classes with weighted multi-dimensional scoring, and show how it instantiates for each use case (Section VI).

II. Background

A. Computing Continuums and Meta-Orchestration

A computing continuum provides a unified abstraction over heterogeneous infrastructure. Unlike traditional orchestrators tied to specific platforms (Kubernetes for cloud, SLURM for HPC), a meta-orchestrator operates *above* platform-specific

schedulers, coordinating workloads across environments without directly controlling the underlying resources.

In ColonyOS, the core abstractions are:

- **Colony:** A distributed runtime environment comprising networked executors.
- **Executor:** A worker process that connects to the colony from any network location, pulls process assignments, executes them, and reports results. Executors can run on cloud VMs, HPC nodes, edge devices, or any Internet-connected system.
- **Process:** A unit of work defined by a function specification, transitioning through states: WAITING → RUNNING → SUCCESS or FAILED.
- **ProcessGraph:** A DAG of processes with dependency edges. Child processes wait until all parent processes complete successfully.
- **WorkflowSpec:** A declarative specification of a ProcessGraph, defining processes and their dependencies by name.

Executors request work by calling `Assign()`, which returns the highest-priority waiting process matching the executor's type and capabilities. This pull-based model means the scheduler's priority function directly determines which work gets done first.

B. DAG Scheduling

DAG scheduling is well-studied in parallel computing [3]. Classical algorithms (HEFT, CPOP) focus on minimizing makespan on a fixed set of processors. Our setting differs in several ways: executors are heterogeneous and dynamically available, DAGs can be modified at runtime (processes inserted between existing nodes), and multiple independent DAGs compete for shared resources with different urgency levels.

C. Current Priority Mechanism

In the current system, each process has an integer `Priority` field set at submission time. The scheduler computes a `PriorityTime` value:

$$\text{PriorityTime} = \text{Priority} \times \Delta t + \text{SubmissionTime} \quad (1)$$

where Δt is a constant scaling factor. The process with the lowest PriorityTime is assigned first. This creates a FIFO ordering within the same priority level, with higher-priority processes jumping ahead. While simple, this mechanism cannot express the multi-dimensional, dynamic, context-dependent priority requirements we identify in our use cases.

III. Use Cases

We present eight use cases that collectively represent the diversity of workloads in computing continuums. Each use case is implemented as a working Go program that constructs and submits a ProcessGraph (DAG) to a ColonyOS instance. We describe the domain context, DAG structure, and priority requirements for each.

A. Use Case 1: Mine Seismology (Continuous Stream)

A seismic monitoring network in the LKAB Kiruna iron ore mine consists of 12 geophones installed at four underground levels (1165m to 1450m depth). The sensors produce a continuous stream of seismogram data via RabbitMQ. For each time window (60 seconds), a processing DAG is submitted with three stages:

1) **P-wave picking** (12 parallel processes): Each geophone's seismogram is analyzed using STA/LTA detection with AIC onset refinement to determine the precise P-wave arrival time. Parameters include bandpass filtering (50–2000 Hz) and minimum SNR thresholds.
2) **Event classification** (1 process, depends on all picks): A CNN-based classifier groups picks by temporal proximity and classifies each group as production blast, development blast, rockburst, or noise. The classifier uses features including pick count, amplitude ratios, frequency content, and spatial distribution.
3) **3D hypocenter location** (N parallel processes, depends on classification): For each classified event, P-wave arrival times are inverted using grid search with Simplex refinement to find the source location (x, y, z) in the mine, using a layered 3D velocity model (P-wave velocities: 5200–6200 m/s across rock types).

A new DAG is submitted every 60 seconds, creating a continuous pipeline. Each DAG contains $12+1+N$ processes, where N is the number of detected events (typically 1–5).

Priority requirements:

- Events detected by more sensors indicate larger seismic events and should be prioritized—this information is only available *after* step 2 completes.
- Certain mine regions (active production levels) are more safety-critical.
- Rockbursts must be prioritized over production blasts.
- The system is safety-critical: delayed rockburst detection can endanger lives.

B. Use Case 2: Satellite Surveillance (Event-Triggered, Multi-User)

Multiple users from different organizations define surveillance jobs, each monitoring a geographic area of interest for environmental changes using Sentinel-2 satellite imagery. When new imagery becomes available for a job's area, a processing DAG is automatically spawned:

1) **Download**: Fetch scene tiles and spectral bands from Copernicus Data Space.
2) **Preprocess**: Cloud masking (using Scene Classification Layer), co-registration against a baseline image, reprojection.
3) **Change detection**: Run the user's chosen algorithm (BFAST Monitor, coherence change, water index thresholding, burn severity, or shoreline extraction) against the baseline.
4) **Notify**: Email, SMS, or webhook notification with change map and statistics.

Five concurrent surveillance jobs are modeled, operated by different Swedish government agencies: illegal logging detection (Länsstyrelsen), highway construction tracking (Trafikverket), flood monitoring (MSB), wildfire scar recovery (Naturvårdsverket), and coastline erosion monitoring (SGU).

Priority requirements:

- Detection type determines urgency: wildfire and flooding alerts must preempt construction tracking and erosion monitoring.
- Each user assigns their own importance level to their surveillance job.
- Multiple jobs may trigger simultaneously from the same satellite pass.
- Jobs from different organizations should be treated fairly.

C. Use Case 3: Agentic AI Workflows (Dynamic DAG)

Users submit natural-language goals to an LLM-powered agentic executor. The system dynamically constructs and extends a DAG as it works:

1) An initial skeleton DAG is submitted: `agent_query` → `llm_plan` → `plan_handler` → `completed`.
2) The plan handler parses the LLM's plan and *inserts* tool steps and a reflect node into the DAG between existing nodes using the `AddChild(insert=true)` mechanism.
3) After tool steps execute, the reflect node evaluates results. If more work is needed, additional steps and another reflect are inserted.
4) This loop continues until the goal is achieved or a maximum iteration count is reached.

Multiple users submit goals concurrently, each producing an independent ProcessGraph that grows dynamically and unpredictably.

Priority requirements:

- DAG size is unknown at submission time—it depends on LLM decisions at runtime.
- User tier should influence priority (researcher vs. student).
- Resource consumption is unpredictable, making fair allocation difficult.

D. Use Case 4: Multi-Tenant ETL Platform (SLA-Driven)

A shared data platform serves three organizations with different service level agreements:

Gold SLA (5-minute deadline): A bank runs real-time fraud detection. The pipeline ingests Kafka transaction streams, enriches with geo-IP and risk scores, matches against sanctions watchlists, scores with an XGBoost ensemble, and dispatches alerts for transactions exceeding fraud thresholds.

Silver SLA (60-minute deadline): A logistics company optimizes daily delivery routes. The pipeline ingests shipment orders and fleet GPS positions, geocodes addresses, solves a vehicle routing problem (VRP) using adaptive large neighborhood search, generates driver manifests, and sends push notifications.

Bronze SLA (24-hour deadline): A university computes weekly bibliometric indicators. The pipeline ingests publications from DiVA, OpenAlex, and Crossref, extracts metadata, builds a citation network, computes h-indices and field-weighted citation impact, and exports results.

All three pipelines compete for the same pool of `etl-worker` executors.

Priority requirements:

- SLA tier determines base priority (Gold > Silver > Bronze).
- Priority should escalate as deadlines approach.
- Bronze should not be completely starved—fairness matters.
- Different pipeline shapes (10 processes vs. 5 processes) compete for the same executors.

E. Use Case 5: Cloud Burst (Meta-Level Infrastructure)

When the seismic processing queue (Use Case 1) builds up beyond on-premise executor capacity, a cloud burst workflow is triggered to scale out:

1) **Assess queue depth**: Check backlog size and wait times.
2) **Estimate capacity**: Calculate VMs needed within budget (EUR 50 max).
3) **Provision VMs**: Create 3 cloud VMs (Hetzner, cpx31).
4) **Install K3s**: Lightweight Kubernetes on each VM.
5) **Deploy executors**: Seismic-processor containers via K3s.
6) **Transfer data**: Velocity model and ML classifier via ColonyFS (parallel with provisioning).
7) **Start**: Register executors with the colony.
8) **Monitor**: Track costs and queue depth.
9) **Teardown**: When the queue drains, delete VMs and unregister executors.

Priority requirements:

- This workflow exists to help *another* workflow—its priority derives from external queue state.
- Creates a feedback loop: burst capacity drains the queue, reducing the burst's own urgency.
- Budget constraints impose a hard ceiling independent of priority.

F. Use Case 6: Cloud-to-HPC LLM Training (Cross-Environment)

A cloud platform user fine-tunes an LLM (Llama 3.1 8B) for Swedish legal text. The workflow spans two environments:

Cloud side: Validate and tokenize 200K legal documents (court rulings, legislation, contracts), download and convert the base model to Megatron format.

Data transfer: Upload tokenized data (85 GB) and model weights (32 GB) to the HPC scratch filesystem via ColonyFS.

HPC side (Dardel, PDC/KTH): Fine-tune with Megatron-LM on 2 nodes $\times$ 4$\times$A100 GPUs using tensor parallelism (TP=4) and pipeline parallelism (PP=2). Evaluate on four Swedish legal benchmarks.

Transfer back: Download trained model, evaluation report, and TensorBoard logs via ColonyFS.

Cloud side: Register in MLflow, deploy with vLLM (OpenAI-compatible API), and validate the endpoint.

Priority requirements:

- The DAG crosses environment boundaries with long data transfers.
- HPC processes may queue in SLURM for hours before executing.
- Two parallel preparation tracks (data + model) converge at training.
- Priority spans two independent scheduling domains.

G. Use Case 7: Drug Discovery Parameter Sweep (Deadline-Driven Fan-Out)

A pharmaceutical company screens 25 drug candidates against SARS-CoV-2 Main Protease using a two-phase funnel:

Phase 1 (wide screening): Prepare the protein target and force field parameters (2 parallel roots). Then dock all 25 compounds with AutoDock Vina (parallel), followed by 25 short molecular dynamics simulations (10 ns each, GPU-accelerated GROMACS). Rank candidates by binding free energy, pose stability, and hydrogen bond analysis.

Phase 2 (deep analysis): Run 100 ns extended MD simulations with replica exchange enhanced sampling for the top 3 candidates. Generate a regulatory submission report.

The workflow has a hard 48-hour deadline for a regulatory committee meeting.

Priority requirements:

- 25 docking + 25 short MD processes compete for GPU resources, prioritized by ML-predicted binding affinity (better candidates first).
- Hard deadline: the final report must be ready in 48 hours.
- Two-phase funnel: phase 2 processes should be higher priority than remaining phase 1 processes.
- Shares HPC GPU resources with other workloads on the same colony.

H. Use Case 8: Water Treatment IoT (Real-Time Edge with Preemption)

A municipal water treatment plant (Norrvatten, serving northern Stockholm) monitors water quality across four treatment stages—raw water intake, coagulation/flocculation, granular activated carbon filtration, and disinfection/distribution—using edge gateways deployed at each stage. Each gateway runs a ColonyOS executor with limited capacity (1–2 concurrent process slots).

Normal operation: Every 30 seconds, a monitoring DAG is submitted for each treatment stage:

1) **Collect sensors**: Read all sensors via OPC-UA from PLCs (turbidity, pH, chlorine residual, pressure, flow, UV transmittance).
2) **Validate readings**: Check values against safe operating ranges, detect flatlines and excessive rate-of-change.
3) **Aggregate metrics**: Compute rolling statistics (30s, 5min, 1h windows) and trend slopes, write to InfluxDB.
4) **Update dashboard**: Push to Grafana operator display.

This produces 4 DAGs $\times$ 4 processes = 16 normal-priority processes every 30 seconds.

Anomaly response: When a sensor breaches a safety threshold (e.g., chlorine residual drops below 0.2 mg/L, filter differential pressure exceeds 35 kPa, or raw water turbidity spikes above 4 NTU), an emergency DAG is submitted:

1) **Emergency collect**: High-rate burst capture (100 Hz for 5 seconds) from all sensors in the affected stage.
2) **Classify severity**: ML classifier determines if the anomaly is critical, warning, or false alarm, cross-validating with neighboring sensors.
3) **Actuate response** and **Alert operators** (parallel): For critical events, send actuator commands (adjust chemical dosing, initiate filter backwash, reduce intake flow) while simultaneously alerting operators via SCADA, SMS, and email.
4) **Notify SCADA**: Update the plant-wide supervisory system and historian.
5) **Log incident**: Record full incident for regulatory compliance (Livsmedelsverket SLVFS 2001:30, 10-year retention).

The emergency DAG has a hard real-time constraint: from anomaly detection to actuator response must complete within 2 seconds. On resource-constrained edge gateways with only 1–2 executor slots, meeting this constraint requires *preemption*—the emergency DAG must interrupt or displace queued and potentially running normal monitoring processes.

Priority requirements:

- **Preemption**: Emergency processes must preempt queued and running normal-priority processes on the same edge gateway. This is not merely queue-jumping; it requires interrupting in-progress work.
- **Hard real-time**: Anomaly-to-actuator latency budget of 2 seconds, with 500 ms maximum wait time for the first emergency process.
- **Mixed criticality**: Safety-critical emergency work and routine monitoring share the same resource-constrained executors.
- **Graduated response**: Critical anomalies (chlorine below safe level) require immediate actuator response; warnings (elevated filter pressure) require operator notification but not immediate actuation.
- **Regulatory compliance**: All incidents must be logged and reported to authorities within 24 hours.

IV. Analysis of Priority Dimensions

Examining the eight use cases reveals that priority is determined by multiple dimensions, some universal and some domain-specific. We classify these dimensions along two axes: *when* the value is known and *where* it comes from.

A. Temporal Classification

Static dimensions have values fixed at submission time: SLA tier (UC4), detection type (UC2), compound predicted binding affinity (UC7), anomaly severity level (UC8).

Dynamic dimensions change value over time without new information: deadline proximity (UC4, UC7), process wait time (all use cases), latency budget remaining (UC8).

Runtime-dependent dimensions only become known when intermediate DAG steps complete: number of triggered sensors (UC1, after classification), candidate ranking (UC7, after short MD), DAG size (UC3, determined by LLM at runtime), confirmed severity after cross-validation (UC8, after classify step).

Externally-dependent dimensions derive from state outside the DAG: queue depth of another workload (UC5), HPC queue wait time (UC6), budget remaining (UC5), current sensor readings breaching thresholds (UC8).

B. Source Classification

Process-intrinsic: Values from the process's own specification—function name, arguments, resource requirements, executor type.

DAG-structural: Values from the process's position in its DAG—depth, number of downstream dependents, DAG completion percentage.

Parent-derived: Values from the output of parent processes—classifier results, intermediate scores.

System-state: Values from the scheduling system—queue depths, executor utilization, elapsed time.

User/tenant metadata: Values from the process submitter—SLA tier, organizational role, budget.

C. Requirements

From the use cases, we derive six requirements for a priority model:

- **R1 Structural safety guarantees.** Safety-critical work (UC1 rockbursts, UC2 wildfire alerts, UC8 anomaly response) must be guaranteed to preempt non-critical work, independent of parameter tuning.
- **R2 Multi-dimensional scoring.** Priority must be expressible as a function of multiple domain-specific dimensions (sensor count, mine region, event type, compound affinity, anomaly severity).
- **R3 Dynamic re-evaluation.** Priority must be recalculable when parent processes complete (UC1), deadlines approach (UC4), or external state changes (UC5).
- **R4 Multi-tenant fairness.** Lower-priority tenants must receive a minimum guaranteed share of resources, preventing starvation (UC4 Bronze tier).
- **R5 Interpretability.** Scheduling decisions must be explainable—critical for safety-critical systems (UC1, UC8) and SLA compliance auditing (UC4).
- **R6 Preemption support.** On resource-constrained executors (UC8 edge gateways), higher-priority processes must be able to preempt running lower-priority processes, not merely jump the queue. This is essential when executor capacity is severely limited and hard real-time deadlines must be met.

V. Alternative Scheduling Methods

We evaluate six approaches against the six requirements identified in Section IV.

A. Weighted Sum Model

The priority of a process p at time t is computed as:

$$\pi(p,t) = \beta(p) + \sum_{i=1}^{n} w_i \cdot d_i(p, G, S, t) \tag{2}$$

where $\beta(p)$ is a base priority, d_i are dimension functions extracting normalized scores from the process p, its DAG G, system state S, and time t, and w_i are user-configured weights.

Strengths: Simple, interpretable, each dimension independently tunable, computationally inexpensive ($O(n)$ per evaluation), supports all dimension sources.

Weaknesses: Assumes dimension independence—cannot express "if region is critical AND sensor count is high, then emergency." More critically, dimensions are *commensurable*: with adversarial or misconfigured weights, a Bronze bibliometrics process can theoretically accumulate enough points to outprioritize a rockburst safety alert. This violates requirement R1.

Assessment: Satisfies R2, R3, R5. Partially satisfies R4 (via an anti-starvation dimension). Fails R1—safety guarantees depend on correct weight configuration rather than structural invariants. Does not address R6 (preemption)—priority ordering alone cannot interrupt running processes.

B. Rule-Based / Production Systems

Priority is determined by an ordered set of condition-action rules:

```
IF event_type = "rockburst"
  AND sensor_count >= 8 -> CRITICAL
IF sla = "Gold"
  AND deadline_remaining < 60s -> CRITICAL
IF sla = "Bronze"       -> LOW
```

Rules are evaluated in order; the first matching rule determines priority.

Strengths: Naturally handles non-linear interactions between dimensions (R2 partially). Highly readable—domain experts can write and audit rules (R5). Can encode structural guarantees by placing safety rules first (R1).

Weaknesses: Rule explosion: as the number of dimensions grows, the number of rules grows combinatorially. A system covering all seven use cases could require hundreds of rules, making the rule set difficult to maintain and verify for completeness. Transitions between priority levels are discrete (cliff effects), not smooth. Difficult to formally analyze properties like starvation freedom.

Assessment: Satisfies R1, R5. Partially satisfies R2 (at the cost of rule explosion). Poorly satisfies R3 (dynamic rules require complex meta-rules). Does not address R4 or R6 directly.

C. Lexicographic / Hierarchical Priority

Priority is a tuple compared lexicographically:

$$\pi(p) = (c_1(p),\ c_2(p),\ \ldots,\ c_k(p)) \tag{3}$$

where c_1 is the most significant component (e.g., urgency class) and c_k the least significant. Comparison proceeds left to right: a higher value in c_i dominates regardless of $c_{i+1}, \ldots, c_k$.

Strengths: Clean semantics—the hierarchy is explicit and verifiable. Safety is structural: if urgency class is c_1, safety-critical work *cannot* be outprioritized by non-critical work regardless of other components (R1). No weight tuning needed for the hierarchy. Used successfully in production systems (Linux CFS nice levels, Kubernetes PriorityClasses).

Weaknesses: Rigid within levels. A Bronze tenant with a 1-second deadline cannot preempt a Gold tenant's background work if SLA tier is a higher-level component than deadline proximity. Escape hatches (e.g., "deadline override promotes urgency class") erode the clean hierarchy. Cannot express smooth trade-offs between dimensions within the same level (R2 partially unsatisfied).

Assessment: Strongly satisfies R1, R5. Partially satisfies R2 (within-level ordering is coarse). Supports R3 if dynamic dimensions can trigger level promotion. Does not directly address R4. Can support R6 if class transitions trigger preemption signals.

D. Constraint-Based Optimization

Scheduling is formulated as an optimization problem:

$$\underset{x}{\text{maximize}} \sum_{p \in P} U(p) \cdot x_p \quad \text{s.t.} \quad \text{resource, fairness, deadline constraints} \tag{4}$$

where $U(p)$ is a utility function and $x_p \in \{0, 1\}$ indicates whether process p is scheduled.

Strengths: Most expressive—can encode all requirements (R1–R5) as constraints. Provably optimal for the given objective. Can reason about trade-offs that other methods cannot ("sacrifice one low-value DAG to meet a deadline on a high-value one").

Weaknesses: Computationally expensive—solving an integer linear program at every scheduling decision does not scale to high-throughput systems (UC1 produces a new DAG every 60 seconds). Requires a global view of all pending work, which is impractical in distributed systems. Scheduling decisions are difficult to explain (R5 violated). The utility function $U(p)$ itself requires the same multi-dimensional modeling we are trying to solve.

Assessment: Satisfies R1–R4 in theory. Can model R6 (preemption as a constraint). Fails R5 (interpretability). Impractical at scheduling-time granularity.

E. Machine Learning Methods

Several ML approaches have been proposed for DAG scheduling:

Reinforcement Learning (RL): An agent observes system state (queue depths, DAG structures, resource utilization) and learns a scheduling policy by optimizing a reward function such as average completion time or deadline compliance. Notable examples include DeepRM [7] for resource management and Decima [8] for DAG scheduling using graph neural networks to encode DAG structure.

Learned priority functions: Instead of hand-tuning weights w_i in Equation 2, the weights are learned from historical scheduling outcomes via supervised learning on (process features, scheduling context, outcome quality) tuples. This automates the weight tuning process while retaining the interpretable weighted sum structure.

Graph Neural Networks (GNNs): GNNs can encode DAG topology and learn which processes to prioritize based on structural properties—critical path length, fan-out width, depth—capturing patterns like "this process is a bottleneck for 25 downstream processes."

Strengths: Can discover non-obvious scheduling strategies. GNN-based approaches (Decima) naturally handle DAG-aware scheduling. Learned priority functions can automate weight tuning. RL can adapt to changing workload patterns over time.

Weaknesses: *Interpretability*: When a rockburst alert is deprioritized and a mine worker is endangered, "the neural network assigned a score of 0.73" is not an acceptable explanation. Black-box scheduling decisions are problematic for safety-critical systems (R5 strongly violated for RL and GNN approaches). *Training data*: RL requires extensive simulation or historical data that may not exist for new workload types. *Generalization*: Models trained on one workload mix may perform poorly when new use cases are added. *Stability*: RL policies can exhibit unexpected behavior in edge cases.

Assessment: Potentially satisfies R2–R4. Fails R1 (safety guarantees are statistical, not structural) and R5 (interpretability). Does not inherently address R6 (preemption decisions require explicit mechanisms, not just score comparisons). Learned weight tuning partially satisfies R5 when combined with a weighted sum structure. Best suited as a complement to interpretable models rather than a replacement.

F. Comparison Summary

Table I summarizes the assessment of each method against the five requirements.

No single method satisfies all six requirements. This motivates a hybrid approach that combines the structural guarantees of lexicographic ordering with the expressiveness of weighted multi-dimensional scoring and explicit preemption support.

VI. Proposed Model

We propose a two-level priority model: lexicographic *urgency classes* provide structural guarantees (R1, R5), while a weighted multi-dimensional *scoring function* within each class provides expressiveness (R2, R3). Fairness constraints (R4) are enforced as scheduling policies orthogonal to priority scoring. Preemption (R6) is triggered by urgency class transitions:

TABLE I
METHOD COMPARISON AGAINST REQUIREMENTS. ● = FULLY SATISFIES, ◐ = PARTIALLY SATISFIES, ○ = DOES NOT SATISFY.

Method	R1 Safety	R2 Multi-dim	R3 Dynamic	R4 Fairness	R5 Interpret.	R6 Preempt.
Weighted Sum	○	●	●	◐	●	○
Rule-Based	●	◐	◐	○	●	○
Lexicographic	●	◐	◐	○	●	◐
Optimization	●	●	●	●	○	◐
ML (RL/GNN)	○	●	●	◐	○	○
ML (Learned w)	○	●	●	◐	◐	○
Hybrid	●	●	●	●	●	●

when a process enters the SAFETY class, it can preempt running processes of lower classes.

A. Urgency Classes

Every process is assigned to exactly one urgency class $C \in \{\text{SAFETY}, \text{DEADLINE}, \text{NORMAL}, \text{BACKGROUND}\}$, ordered by strict dominance:

$$\text{SAFETY} \succ \text{DEADLINE} \succ \text{NORMAL} \succ \text{BACKGROUND} \quad (5)$$

A process in a higher class is *always* scheduled before a process in a lower class, regardless of within-class scores. Class assignment is determined by a set of *promotion rules*:

$$C(p,t) = \max_{r \in R} \{c_r \mid \phi_r(p, G, S, t) = \text{true}\} \quad (6)$$

where R is the set of promotion rules, c_r is the class that rule r promotes to, and ϕ_r is a boolean predicate. The default class is NORMAL.

Promotion rules are intentionally simple boolean predicates, not learned functions:

- SAFETY: `event_type = ``rockburst''` $\wedge$ `sensor_count` $\geq 6$ (UC1); `detection_type = ``wildfire''` $\wedge$ `change_detected` (UC2); `anomaly_severity = ``critical''` (UC8).
- DEADLINE: `deadline_remaining` $< 0.1 \times$ `deadline_total` (UC4, UC7)—any process within 10% of its deadline; `latency_budget_remaining` $<$ `500ms` (UC8).
- BACKGROUND: `sla = ``Bronze''` $\wedge$ `no_deadline` (UC4); `label = ``best-effort''`.

The key property is that these rules are *auditable*: a human can inspect the rule set and verify that safety-critical work will always be classified correctly. This is a structural guarantee, not a statistical one.

B. Within-Class Scoring

Within each urgency class, processes are ordered by a weighted scoring function:

$$\sigma(p,t) = \beta(p) + \sum_{i=1}^{n} w_i \cdot d_i(p, G, S, t) \quad (7)$$

where $\beta(p)$ is a base score, w_i are configurable weights, and d_i are *dimension functions* that extract a normalized value $d_i \in [0,1]$ from the available context. The process with the highest score is scheduled first.

1) Dimension Functions: A dimension function d_i maps a process and its context to a normalized score. We define a taxonomy of dimension sources:

- $d_k^{\text{kwarg}}(p) = f(\text{kwarg}(p,k))$: Extract from the process's keyword arguments and apply a transform f (categorical mapping, linear scaling, etc.).
- $d_k^{\text{parent}}(p, G) = f(\text{output}(\text{parent}(p,G), k))$: Extract from a parent process's output in the DAG. Only available after the parent completes; evaluates to a default value otherwise.
- $d^{\text{dag}}(p, G) = |\{q \in G : \text{state}(q) = \text{SUCCESS}\}|/|G|$: DAG completion fraction. Captures the "momentum" of a partially completed DAG.
- $d^{\text{wait}}(p,t) = \min(1, (t - t_{\text{submit}}(p))/\tau)$: Normalized wait time with saturation constant τ. Provides anti-starvation within a class.
- $d^{\text{deadline}}(p,t) = 1 - \min(1, (t_{\text{deadline}}(p) - t)/t_{\text{total}}(p))$: Deadline proximity, from 0 (just submitted) to 1 (at deadline).
- $d_k^{\text{system}}(S) = f(\text{state}(S,k))$: Extract from system state (queue depth, budget remaining, executor utilization).

2) Transforms: The transform function f normalizes raw values to $[0,1]$:

- **Linear:** $f(x) = \min(1, (x - x_{\min})/(x_{\max} - x_{\min}))$
- **Categorical:** $f(x) = m(x)$ where m is a user-defined mapping
- **Exponential:** $f(x) = 1 - e^{-x/\lambda}$ for deadline urgency
- **Logarithmic:** $f(x) = \min(1, \log(1+x)/\log(1+x_{\max}))$ for anti-starvation (slow initial growth)

C. Re-evaluation Triggers

Priority is not computed once at submission time. The effective priority $\Pi(p,t) = (C(p,t), \sigma(p,t))$ is re-evaluated on the following events:

1) **Parent completion:** When a parent process completes, its output becomes available, enabling parent-derived dimensions (d^{parent}) and potentially triggering class promotion.
2) **Timer tick:** Periodic re-evaluation (configurable interval) updates time-dependent dimensions (d^{wait}, d^{deadline}) and can trigger deadline-based promotion to the DEADLINE class.
3) **System state change:** Significant changes in queue depth or executor availability trigger re-evaluation of system-dependent dimensions (d^{system}).

D. Fairness Constraints

Fairness is enforced as a constraint on the scheduling policy, orthogonal to priority scoring. We define minimum share guarantees per tenant:

$$\forall j \in \text{Tenants} : \frac{\text{assigned}(j, \Delta t)}{\text{total_assigned}(\Delta t)} \geq s_j \tag{8}$$

where s_j is the minimum share for tenant j and Δt is a sliding window. When a tenant falls below its minimum share, its next process is scheduled regardless of priority score, temporarily overriding the priority ordering.

This separation is deliberate: priority determines ordering *within* the fairness constraint, but fairness provides a floor that priority cannot erode.

E. Preemption

When a process is promoted to the SAFETY class and the target executor has no idle slots, the scheduler must preempt a running lower-class process. We define preemption as a two-step mechanism:

1) **Preemption signal:** The scheduler identifies the lowest-priority running process on the target executor (or executor type) and sends a preemption signal. The executor suspends the running process, saving its state if possible (checkpoint), and marks it as WAITING with a `preempted` flag.
2) **Resumption:** Once the SAFETY-class process completes, the preempted process is resumed from its checkpoint (if available) or restarted.

Preemption is only triggered across urgency class boundaries—never within the same class. This prevents cascading preemptions and ensures that preemption overhead is bounded. In practice, preemption is most relevant on resource-constrained edge executors (UC8), where the alternative—waiting for a normal-priority process to complete—would violate hard real-time constraints.

The preemption policy is configurable per executor type:

- **Preemptive:** Edge gateways (UC8) allow preemption of NORMAL and BACKGROUND processes by SAFETY processes.
- **Non-preemptive:** Cloud and HPC executors with abundant capacity rely on queue-jumping (the default behavior of urgency classes) without interrupting running work.
- **Cooperative:** Executors that support checkpointing can save progress before yielding, reducing wasted work.

F. DAG Momentum

To avoid wasting work invested in partially completed DAGs, we include a DAG completion dimension d^{dag} with a positive weight. This means a process belonging to a 90%-complete DAG receives a priority boost over an equivalent process in a newly submitted DAG. This captures the intuition that completing existing work is more valuable than starting new work, without requiring a separate mechanism.

G. Effective Priority

The complete effective priority is a tuple:

$$\Pi(p, t) = \big(C(p, t),\ \sigma(p, t)\big) \tag{9}$$

compared lexicographically. The scheduler selects the waiting process with the highest Π that matches the requesting executor's type and capabilities.

H. Instantiation Examples

We illustrate how the model is instantiated for three representative use cases.

1) UC1: Mine Seismology.

Listing 1. Priority model for seismic processing.

```
model: seismic_processing
base: 10

promotion_rules:
  SAFETY:
    - event_type = "rockburst"
      AND sensor_count >= 6

dimensions:
  - name: sensor_count
    source: parent_output("classify", "count")
    transform: linear(min=3, max=12)
    weight: 5.0

  - name: mine_region
    source: kwarg("region")
    transform: categorical(
      production_L1252: 1.0,
      development_L1450: 0.7,
      exploration: 0.3)
    weight: 3.0

  - name: event_type
    source: parent_output("classify", "class")
    transform: categorical(
      rockburst: 1.0,
      blast_production: 0.5,
      blast_development: 0.3)
    weight: 4.0
```

Note that `sensor_count` and `event_type` are parent-derived dimensions: they are only available after the classification step completes. Before that, the process uses the base priority. When the parent completes, the process's priority is re-evaluated, potentially promoting it to the SAFETY class.

2) UC4: Multi-Tenant ETL.

Listing 2. Priority model for multi-tenant ETL.

```
model: tenant_pipeline
base: 0

promotion_rules:
  DEADLINE:
    - deadline_remaining < 0.1 * deadline_total

dimensions:
  - name: sla_tier
    source: kwarg("sla")
    transform: categorical(
      Gold: 1.0, Silver: 0.5, Bronze: 0.15)
    weight: 15.0

  - name: deadline_urgency
    source: deadline_proximity()
    transform: exponential(halflife=300s)
    weight: 20.0

  - name: wait_time
    source: wait_time()
    transform: logarithmic(max=3600s)
    weight: 2.0
```

```
fairness:
  method: weighted_fair_queue
  min_share:
    tenant-a: 0.50
    tenant-b: 0.30
    tenant-c: 0.10
```

The logarithmic transform on wait time provides anti-starvation within the NORMAL class: it grows quickly at first (preventing short starvation) then saturates (preventing unbounded priority growth that could invert SLA ordering).

3) UC7: Drug Discovery.

Listing 3. Priority model for drug discovery sweep.

```
model: drug_screening
base: 10

promotion_rules:
  DEADLINE:
    - deadline_remaining < 4h

dimensions:
  - name: predicted_binding
    source: kwarg("predicted_binding")
    transform: linear(min=-12, max=-3)
    weight: 6.0

  - name: phase
    source: kwarg("phase")
    transform: categorical(
      screening: 0.3, deep_analysis: 1.0)
    weight: 5.0

  - name: dag_progress
    source: dag_completion()
    transform: linear(min=0, max=1)
    weight: 3.0

  - name: deadline_urgency
    source: deadline_proximity()
    transform: exponential(halflife=6h)
    weight: 8.0
```

Here, `predicted_binding` ensures compounds with better predicted affinity get GPU time first. The `phase` dimension ensures that phase 2 (deep analysis) processes for the top 3 candidates are prioritized over remaining phase 1 screening processes. The `dag_progress` dimension provides momentum: as the sweep nears completion, the final report process receives a boost.

4) UC8: Water Treatment IoT.

Listing 4. Priority model for IoT edge with preemption.

```
model: water_treatment_iot
base: 0

promotion_rules:
  SAFETY:
    - anomaly_severity = "critical"
  DEADLINE:
    - latency_budget_remaining < 500ms

dimensions:
  - name: anomaly_severity
    source: kwarg("severity")
    transform: categorical(
      critical: 1.0, warning: 0.6, info: 0.1,
      normal_monitoring: 0.0)
    weight: 10.0

  - name: sensor_deviation
    source: kwarg("trigger_value") / kwarg("
        threshold")
    transform: linear(min=1.0, max=5.0)
    weight: 4.0

  - name: treatment_stage
    source: kwarg("stage")
    transform: categorical(
      disinfection: 1.0,
      filtration: 0.7,
      coagulation: 0.5,
      raw_intake: 0.3)
    weight: 3.0

  - name: latency_remaining
    source: latency_budget()
    transform: exponential(halflife=500ms)
    weight: 8.0

preemption:
  executor_type: edge-iot
  policy: preemptive
  preempt_classes: [NORMAL, BACKGROUND]
  checkpoint: cooperative_if_supported
```

The `anomaly_severity` dimension, combined with the SAFETY promotion rule, ensures that critical anomalies immediately preempt normal monitoring on the edge gateway. The `latency_remaining` dimension with exponential transform creates increasing urgency as the 2-second real-time budget is consumed. The preemption policy is configured specifically for the `edge-iot` executor type, reflecting the resource-constrained nature of edge gateways.

VII. DISCUSSION

A. Composability

A key advantage of the proposed model is that priority models compose naturally when multiple workload types share a colony. Each workload type defines its own dimensions and weights, but the urgency classes provide a shared framework for inter-workload ordering. A colony administrator defines the promotion rules and fairness constraints, while workload developers define the within-class scoring.

B. Relationship to Classical Scheduling

The model subsumes several classical scheduling policies as special cases:

- **FIFO:** No dimensions, equal base priority.
- **Earliest Deadline First (EDF):** One dimension d^{deadline}, weight 1.
- **Shortest Job First (SJF):** One dimension based on estimated execution time, weight 1.
- **Fair Share:** Fairness constraints with equal minimum shares and no within-class scoring.

This suggests the model is sufficiently general to serve as a unifying framework.

C. Role of Machine Learning

While we argued against ML-based scheduling as a standalone approach (due to interpretability concerns in safety-critical settings), ML can complement the proposed model in two ways:

Weight tuning: The dimension weights w_i can be learned from historical scheduling outcomes rather than hand-tuned. Since the model structure (dimensions and urgency classes) remains interpretable, this preserves R5 while automating parameter selection.

Dimension prediction: ML models can predict dimension values that are not yet available. For example, in UC1, a model could predict the likely sensor count before the classification step completes, enabling earlier priority adjustment. This is a prediction within an interpretable framework, not a black-box scheduling decision.

D. Preemption Trade-offs

Use Case 8 demonstrates that queue-ordering alone is insufficient on resource-constrained executors. When an edge gateway has only one executor slot and it is occupied by a normal monitoring process, a safety-critical anomaly response cannot wait. Preemption introduces costs—suspended work may need to be restarted, checkpointing adds overhead, and preemption cascades must be prevented—but these costs are acceptable when the alternative is violating hard real-time safety constraints.

Our model limits preemption to cross-class boundaries (SAFETY preempts NORMAL/BACKGROUND), which bounds the preemption frequency. In practice, SAFETY-class events are rare (anomalies in a well-maintained plant), so preemption overhead is low in aggregate. The configurable preemption policy per executor type allows operators to disable preemption on executors where it is unnecessary (cloud, HPC) or enable cooperative checkpointing where supported.

A key open question is the interaction between preemption and DAG momentum: if a process in a 90%-complete normal DAG is preempted, should the DAG's momentum bonus apply when the preempted process is rescheduled? We argue yes—the investment in partially completed DAGs should not be penalized by external events.

E. Inter-Workflow Dependencies

Use Case 5 (cloud burst) illustrates a pattern where one workflow's priority derives from another workflow's state—the burst workflow's urgency depends on Use Case 1's queue depth. Our model handles this through system-state dimensions (d^{system}), but it does not formally model *inter-DAG dependencies*: the case where Workflow B cannot start until Workflow A produces a specific output.

In the current model, such dependencies must be encoded externally (e.g., a monitoring process that polls for Workflow A's completion and then submits Workflow B). A richer model could support first-class inter-DAG edges, where a process in one DAG depends on a process in another DAG. This would enable priority propagation across DAG boundaries—if Workflow B is high-priority, Workflow A's blocking process should inherit some of that urgency. Formalizing this extension without introducing circular dependencies or unbounded priority propagation is an open problem.

F. Failure Recovery and Retry Priority

None of the eight use cases explicitly address what happens when a process fails and must be retried. In practice, a failed process in a partially completed DAG presents a priority dilemma:

- **Boost priority:** The failed process is now on the critical path of a DAG that has already consumed resources. The DAG momentum dimension naturally provides some boost, but a failed process may warrant additional urgency to avoid wasting the investment in completed upstream processes.
- **Reduce priority:** If the process failed due to a transient issue (resource exhaustion, network timeout), it may fail again immediately. Reducing priority gives the system time to recover.
- **Escalate class:** If a retried process causes its DAG to approach a deadline, the deadline promotion rule should trigger automatically via re-evaluation.

Our model's re-evaluation mechanism (Section VI) handles the deadline case naturally: when a failure delays a DAG, the remaining processes' deadline proximity increases, potentially promoting them to the DEADLINE class. However, the model does not currently distinguish between a first attempt and a retry. A *retry count* dimension could be added to the scoring function, with a weight that could be positive (boost retries to protect DAG investment) or negative (penalize repeated failures), depending on the deployment's failure semantics.

G. Limitations

The model assumes that urgency classes can be cleanly separated for a given deployment. In practice, the boundary between SAFETY and DEADLINE may be domain-dependent and politically contested. The model also requires explicit configuration—promotion rules, dimension definitions, weights, and fairness shares—which imposes an operational burden on system administrators.

The preemption mechanism introduces complexity in executor implementations: executors must support process suspension and, ideally, checkpointing. Not all workloads are preemptible—a GPU kernel mid-execution or a database transaction in progress cannot be cleanly interrupted. The model's per-executor-type preemption policy mitigates this, but operators must correctly classify which executor types support preemption.

The fairness mechanism (minimum shares per tenant) is a simple proportional guarantee. More sophisticated fairness notions (e.g., max-min fairness, dominant resource fairness [6]) could be integrated but are beyond the scope of this work.

VIII. RELATED WORK

DAG scheduling in clusters. HEFT [4] and CPOP prioritize tasks by upward rank (critical path length). These assume a fixed processor set and optimize makespan for a single DAG. Our work addresses multi-DAG scheduling on dynamic, heterogeneous executors with diverse priority requirements.

Multi-criteria scheduling. Braun et al. [5] compare heuristics for heterogeneous computing. Most multi-criteria approaches target makespan and resource utilization; we focus on priority as a user-facing concern with safety and fairness requirements.

Fair scheduling. Dominant Resource Fairness (DRF) [6] provides max-min fairness for multi-resource environments. Our fairness constraints are simpler (proportional shares) but interact with a richer priority model.

RL for scheduling. DeepRM [7] and Decima [8] apply deep RL to resource management and DAG scheduling respectively. These achieve strong performance but sacrifice interpretability, which we argue is essential for safety-critical deployments.

Workflow management systems. Apache Airflow, Prefect, and Dagster support DAG-based workflows with basic priority levels. None provide multi-dimensional dynamic priority or cross-DAG priority dependencies.

Computing continuums. Dustdar et al. [1] survey the computing continuum vision. Our work contributes a concrete scheduling model for the heterogeneous workloads that arise in these environments.

IX. Conclusion

We presented eight diverse workload use cases in computing continuums and demonstrated that single-integer priority is insufficient for scheduling heterogeneous DAG workloads. Through systematic analysis, we identified six requirements—structural safety guarantees, multi-dimensional scoring, dynamic re-evaluation, multi-tenant fairness, interpretability, and preemption support—and showed that no existing method satisfies all six.

Our hybrid priority model combines lexicographic urgency classes with weighted multi-dimensional scoring. The urgency classes provide auditable structural guarantees for safety-critical and deadline-sensitive work, and serve as the trigger for preemption on resource-constrained edge executors. The weighted scoring function provides expressive, domain-specific ordering within each class. Fairness constraints prevent starvation independently of priority scores. The model is re-evaluated on parent completion, timer ticks, and system state changes, supporting the dynamic and runtime-dependent priority dimensions that arise in practice.

The model is general—subsumes classical scheduling policies as special cases—yet practical: each component (promotion rules, dimension functions, transforms, weights, fairness shares, preemption policies) is independently configurable and interpretable. We identified inter-workflow dependencies and failure recovery as open problems that merit further investigation. Future work includes implementation in ColonyOS, empirical evaluation with the eight workload types running concurrently, investigation of learned weight tuning as a complement to manual configuration, and formal analysis of preemption overhead on edge deployments.

References

[1] S. Dustdar, V. Casamayor Pujol, and P. K. Donta, "On distributed computing continuum systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 4, pp. 4092–4105, 2023.

[2] ColonyOS, "ColonyOS – Distributed meta-orchestrator," 2024. [Online]. Available: https://github.com/colonyos/colonies

[3] Y.-K. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Computing Surveys*, vol. 31, no. 4, pp. 406–471, 1999.

[4] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.

[5] T. D. Braun *et al.*, "A comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems," *Journal of Parallel and Distributed Computing*, vol. 61, no. 6, pp. 810–837, 2001.

[6] A. Ghodsi, M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica, "Dominant resource fairness: Fair allocation of multiple resource types," in *Proc. NSDI*, 2011, pp. 323–336.

[7] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. HotNets*, 2016, pp. 50–56.

[8] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, "Learning scheduling algorithms for data processing clusters," in *Proc. SIGCOMM*, 2019, pp. 270–288.